

Text Preprocessing and NLP Steps



Natural Language Processing (NLP): Definition



Natural language processing is the ability to take a body of text and **extract meaning** from it using a computer.

Understanding Language

Video 4:22 minutes



Natural Language Challenge: Human vs Computer

HUMAN

“I drove my friend Mary to the park in my Tesla while listening to music on my iPhone.”



Humans understand that Mary is a friend and that a Tesla is likely a car. Additionally, after many years of popularity and cultural references, we all know that an iPhone is a smartphone.

MACHINE

Structured data

`<friend>Mary</friend>
<car>Tesla</car>
<phone>iPhone</phone>`

None of this is understood by a computer without assistance.



NLP Components

Let's analyze the phrase ...

"I drove my friend Mary to the park in my Tesla while listening to music on my iPhone"

Entities

The people, places, organizations, and things in your text.

Example: friend, car, and phone

Semantic Roles

Subjects, actions, and objects in the text.

"IBM bought a company." The subject is "IBM," the action is "bought," and the object is "company."

Relations

How entities are related.

A "createdBy" relation might connect the entities "iPhone" and "Apple."

Categories

Describing what a piece of content is about at a high level.

Categories could be sports, finance, travel, computing, and so on.

Concepts

Extracting reference to topics that do not explicitly appear in the text.

An article about Tesla may refer to concepts "electric cars" or "Elon Musk," even if those terms are not explicitly mentioned.

Emotion

Understanding the emotion or tone conveyed.

Is the content conveying anger, disgust, fear, joy, or sadness?

Keywords

Identify the important and relevant keywords in your content.

Sentiment

Is the feeling/attitude positive, neutral, or negative?

The level of positive or negative sentiment can be scored.

Natural Language Processing

The semantic behind
the syntactic

Syntactic messages

Subject-verb-object

Semantic messages

Agent and Patient

What is the message sentiment?



* Search for 'moving' at <http://www.shoecomics.com/>

Natural language processing: a classification problem

- Difficulty of language: Subtleties, idiosyncrasies, idioms, ambiguities, nuances and gaps

Noses run	but	feet smell
Slim chance	=	Fat chance
Wise man	not	Wise guy
Alarm goes off	while	it goes on
Fill in a form	by	filling it out
House burns up	while	it burns down
Ship by truck	but	send cargo by ship

- It is highly contextual, imprecise and has gaps (context known outside the conversation)

NLP Steps

Segmentation

Tokenization

Removing Stop Words

Stemming

Lemmatization

Part of Speech Tagging

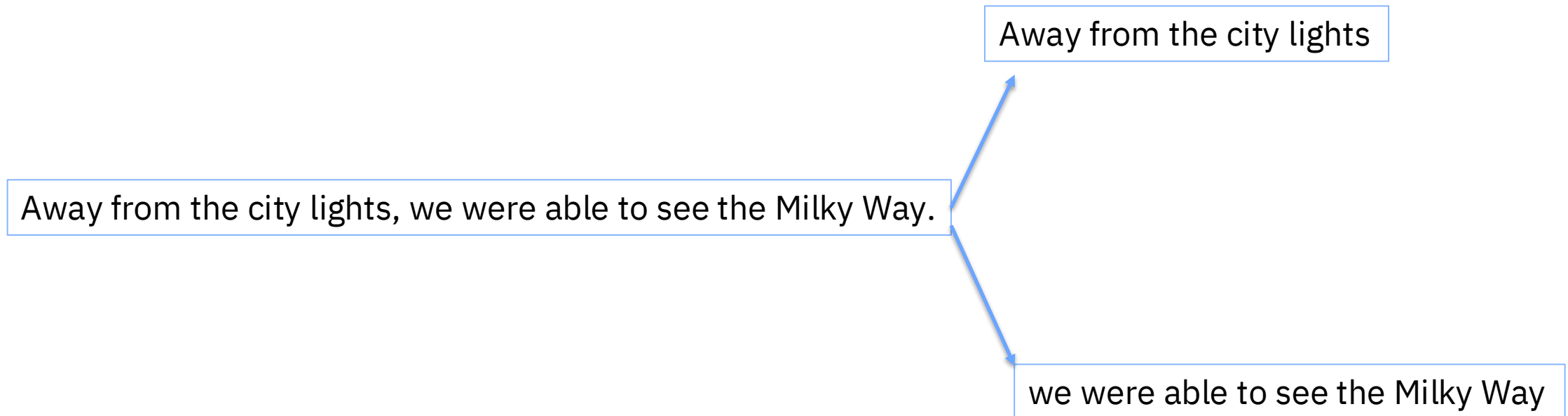
Named Entity Detection

Full stack



Task 1: Segmentation

The first action is to break down the entire text into sentences, snippets, noun phrases, most of which are already denoted by commas, semicolons, periods, and other punctuation.



Segmentation Example

```
import nltk
from nltk.tokenize import sent_tokenize

# Download the tokenizer models (only once)
nltk.download('punkt')

# Sample paragraph
text = "Away from the city lights, we were able to see the Milky Way."

# Split into sentences
sentences = sent_tokenize(text)

# Show the segmented sentences
print("Segmented Sentences:")
for i, sentence in enumerate(sentences, 1):
    print(f"{i}. {sentence}")
```

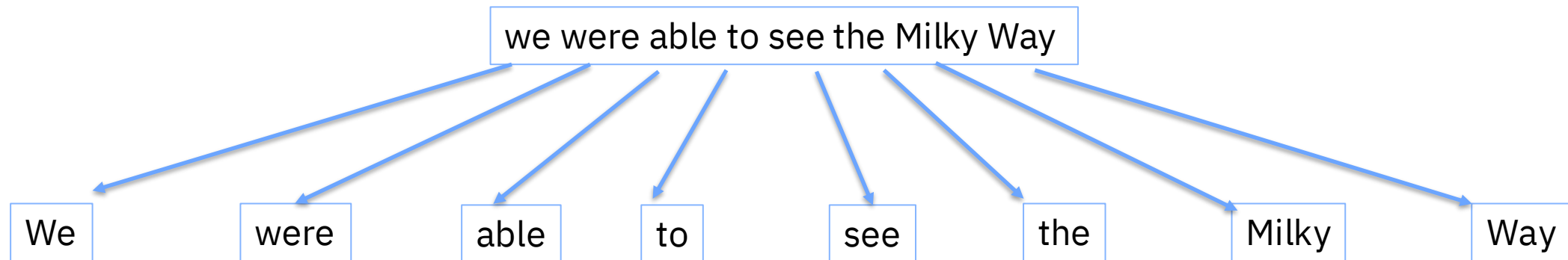
Segmented Sentences:

1. Away from the city lights,
2. we were able to see the Milky Way.

Task 2: Tokenization

To help the algorithm understand a sentence, we first need to break it down into individual words and look at each one separately.

This process is called *tokenizing*, and each word we extract is known as a *token*. By doing this, we make it easier for the algorithm to process and analyze the sentence.



Tokenized

```
import nltk
from nltk.stem import PorterStemmer, WordNetLemmatizer
from nltk.tokenize import word_tokenize

# Download required data
nltk.download('punkt')
nltk.download('wordnet')
nltk.download('omw-1.4')

# Initialize stemmer and lemmatizer
stemmer = PorterStemmer()
lemmatizer = WordNetLemmatizer()

# Example sentence
sentence = "The runners were running and caring about their connections."

# Tokenize the sentence
tokens = word_tokenize(sentence)

# Apply stemming
stemmed = [stemmer.stem(word) for word in tokens]

# Apply lemmatization
lemmatized = [lemmatizer.lemmatize(word) for word in tokens]

# Print results
print("Original      :", tokens)
print("Stemmed       :", stemmed)
print("Lemmatized     :", lemmatized)
```

Original : ['The', 'runners', 'were', 'running', 'and', 'caring', 'about', 'their', 'connections', '.']
Stemmed : ['the', 'runner', 'were', 'run', 'and', 'care', 'about', 'their', 'connect', '.']
Lemmatized : ['The', 'runner', 'were', 'running', 'and', 'caring', 'about', 'their', 'connection', '.']

Task 3: Removing the Stop Words

To speed up the learning process, we remove common words that don't add much meaning to a sentence. These words—like *from*, *the*, *to*, *the*, and *and*—are called *stop words*. Removing them helps the algorithm focus on the essential parts of the text.

```
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize

# Download stopwords (only needed once)
nltk.download('punkt')
nltk.download('stopwords')

# Sample sentence
text = "Away from the city lights, we were able to see the Milky Way"

# Step 1: Tokenize the sentence
tokens = word_tokenize(text)

# Step 2: Remove stop words
stop_words = set(stopwords.words('english'))
filtered_tokens = [word for word in tokens if word.lower() not in stop_words]

# Print results
print("Original tokens:", tokens)
print("Filtered tokens (no stop words):", filtered_tokens)
```

```
Original tokens: ['Away', 'from', 'the', 'city', 'lights', ',', 'we', 'were', 'able', 'to', 'see', 'the', 'Milky', 'Way']
Filtered tokens (no stop words): ['Away', 'city', 'lights', ',', 'able', 'see', 'Milky', 'Way']
```

Task 4: Stemming

Stemming is a common NLP technique that reduces a word to its **base or root form** (called the *stem*), often by chopping off suffixes. It helps group together words with similar meanings so they can be analyzed as one.

```
import nltk
from nltk.stem import PorterStemmer
from nltk.tokenize import word_tokenize

# Download necessary data if running for the first
time
nltk.download('punkt')

# Create stemmer
stemmer = PorterStemmer()

# Example sentence
sentence = "we were able to see the Milky Way"

# Tokenize sentence
tokens = word_tokenize(sentence)

# Stem each word
stemmed_words = [stemmer.stem(word) for word in
tokens]

# Show results
print("Original words:", tokens)
print("Stemmed words :", stemmed_words)
```



Stemming is fast and simple, but it sometimes **over-stems** (like *able* → *abl* or *Milky* → *milki*),

which is why **lemmatization** is sometimes preferred when accuracy matters.

Original words: ['we', 'were', 'able', 'to', 'see', 'the', 'Milky', 'Way']

Stemmed words : ['we', 'were', 'abl', 'to', 'see', 'the', 'milki', 'way']

Task 5: Lemmatization

Lemmatization is the process of finding the **base or dictionary form** of a word, known as the **lemma**.

- It turns words like **"running"**, **"ran"**, or **"runs"** into **"run"**
- The lemma is always a real word you'd find in a dictionary
- It uses **grammar rules** (like tense, mood, or number) to choose the correct base form

For example:

"better" → **"good"** (because it knows "better" is a comparative form)

```
Original : ['we', 'were', 'able', 'to', 'see', 'the', 'Milky', 'Way']
Stemmed  : ['we', 'were', 'abl', 'to', 'see', 'the', 'milki', 'way']
Lemmatized : ['we', 'were', 'able', 'to', 'see', 'the', 'Milky', 'Way']
```

```
import nltk
from nltk.stem import PorterStemmer, WordNetLemmatizer
from nltk.tokenize import word_tokenize

# Download required data
nltk.download('punkt')
nltk.download('wordnet')
nltk.download('omw-1.4')

# Initialize stemmer and lemmatizer
stemmer = PorterStemmer()
lemmatizer = WordNetLemmatizer()

# Example sentence
sentence = "we were able to see the Milky Way"

# Tokenize the sentence
tokens = word_tokenize(sentence)

# Apply stemming
stemmed = [stemmer.stem(word) for word in tokens]

# Apply lemmatization
lemmatized = [lemmatizer.lemmatize(word) for word in tokens]

# Print results
print("Original      :", tokens)
print("Stemmed       :", stemmed)
print("Lemmatized     :", lemmatized)
```



Task 6: POS: Part of Speech

Every word in English sentences fall into nine major POS types. They are

adjectives,

verbs,

pronouns,

conjunctions,

prepositions,

articles (determiners),

adverbs,

nouns, and

interjections

Part-of-Speech (POS) Tagging means labeling each word in a sentence with its role or function. It answers the following questions:

What kind of word is this?

- Is it a **noun** (thing)?
- A **verb** (action)?
- An **adjective** (description)?

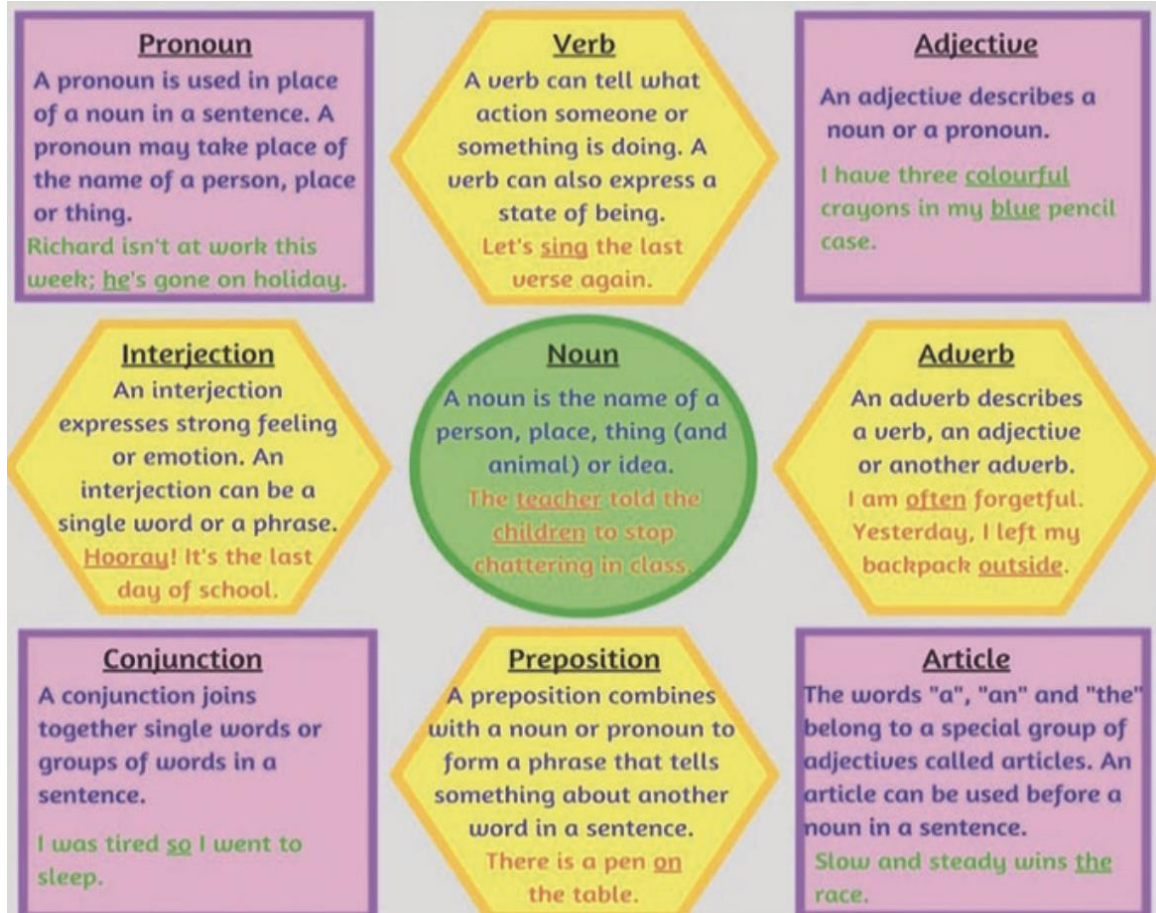
Example:

The quick brown fox jumps over the lazy dog.

→ “The” = determiner, “fox” = noun, “jumps” = verb

Example of POS tagging

Wow! The excited dog quickly ran across the yard because it saw another cat.



<u>Word</u>	<u>Part of Speech</u>	<u>Why?</u>
Wow	Interjection	Expresses emotion/surprise
The	Article (Determiner)	Points to a specific noun
excited	Adjective	Describes the noun "dog"
dog	Noun	Person/place/thing
quickly	Adverb	Describes how the dog ran
ran	Verb	Action word
across	Preposition	Shows relationship (direction)
the	Article (Determiner)	Again points to a specific noun
yard	Noun	Place
because	Conjunction	Connects clauses
it	Pronoun	Replaces "dog"
saw	Verb	Action (past tense)
another	Adjective	Describes "cat"
cat	Noun	Thing

Example of POS Tagging

Run this code in Anaconda:

```
import nltk
from nltk import word_tokenize, pos_tag

# Download necessary data (only the first time)
nltk.download('punkt')
nltk.download('averaged_perceptron_tagger')

# Sample sentence
sentence = "Wow! The excited dog quickly ran across the yard because it saw another cat."

# Step 1: Tokenize the sentence
tokens = word_tokenize(sentence)

# Step 2: POS tagging
tagged = pos_tag(tokens)

# Output results
print("POS Tagging Results:")
for word, tag in tagged:
    print(f"{word:10} -> {tag}")
```



POS Tagging Results:

Wow	-> NN
!	-> .
The	-> DT
excited	-> JJ
dog	-> NN
quickly	-> RB
ran	-> VBD
across	-> IN
the	-> DT
yard	-> NN
because	-> IN
it	-> PRP
saw	-> VBD
another	-> DT
cat	-> NN
.	-> .



You'll see tags like:

- NN – Noun
- VB – Verb (base)
- VBD – Verb (past tense)
- JJ – Adjective
- RB – Adverb
- DT – Determiner
- IN – Preposition
- PRP – Pronoun
- UH – Interjection
- CC – Conjunction

Task 7: Named Entity Recognition

What is Named Entity Recognition (NER)?

Named Entity Recognition (NER) is a process in NLP where we identify and classify proper nouns in a sentence into predefined categories, such as:

-  Person (e.g., "Barack Obama")
-  Location (e.g., "New York")
-  Organization (e.g., "Google")
-  Date/Time (e.g., "January 1st", "next week")
-  Money/Percentage (e.g., "\$1,000", "20%")

Why is NER Important?

NER helps machines understand real-world entities in text, making it essential for:

-  Email filtering
-  News categorization
-  Legal and medical document analysis
-  Chatbots and digital assistants
-  Search engines and summarization tools

It's how a computer knows that "Amazon" is a company, not just a forest.

Task 8 - Custom Text Preprocessing (Tokenization + Stopword Removal + Lemmatization)

This exercise is provided by Suman Attri, Saint Peter's University, NJ. 2026

```
import nltk
import string
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords, wordnet
from nltk.stem import WordNetLemmatizer

# Download necessary NLTK data (only first time)
nltk.download('punkt')
nltk.download('stopwords')
nltk.download('wordnet')
import nltk
nltk.download('averaged_perceptron_tagger_eng')

# Helper function to map NLTK POS tags to WordNet POS tags
def get_wordnet_pos(treebank_tag):
    if treebank_tag.startswith('J'):
        return wordnet.ADJ
    elif treebank_tag.startswith('V'):
        return wordnet.VERB
    elif treebank_tag.startswith('N'):
        return wordnet.NOUN
    elif treebank_tag.startswith('R'):
        return wordnet.ADV
    else:
        return wordnet.NOUN # default to noun

# continue with copy/pasting the following code from the next page...
```

Continues form previous page

```
# Tokenize function
def tokenize(text, remove_stopwords=True, normalize='lemmatize'):
    # Lowercase
    text = text.lower()

    # Tokenize
    tokens = word_tokenize(text)

    # Remove punctuation
    tokens = [w for word in tokens for w in word.strip(string.punctuation).split()]

    # Remove stopwords
    if remove_stopwords:
        stop_words = set(stopwords.words('english'))
        tokens = [w for w in tokens if w not in stop_words]

    # Lemmatize or stem
    if normalize == 'lemmatize':
        lemmatizer = WordNetLemmatizer()
        tagged_tokens = nltk.pos_tag(tokens)
        tokens = [lemmatizer.lemmatize(word, get_wordnet_pos(pos))
                   for word, pos in tagged_tokens]

    return tokens

# Test sentence
sentence = "The children were running faster and performed better while studying different programming languages"

cleaned_sentence = tokenize(sentence, remove_stopwords=True, normalize='lemmatize')
print(cleaned_sentence)
```

Resources

1. Beginner Guide to Natural Language Processing
<https://developer.ibm.com/articles/a-beginners-guide-to-natural-language-processing/>
2. IBM Project Debater
https://youtu.be/3_yy0dnIc58

What is NLP (Natural Language Processing)?

<https://www.youtube.com/watch?v=fLvJ8VdHLA0>

NLP vs NLU vs NLG

<https://www.youtube.com/watch?v=1l6bQ12VxV0>

The History of Natural Language Processing (NLP)

<https://www.youtube.com/watch?v=Uuz8ZTV5vdA>

Types of Natural Language Processing (NLP)

https://www.youtube.com/watch?v=oxzhX_XaEl8